

LogiCube



Arc à flèche

Télécharger le pack depuis GitHub

Télécharger le pack d'asset sur git

[https://github.com/Ayly-
EXE/LogiCube/blob/Cours8/Bow/Voxel%20
Weapons.zip](https://github.com/Ayly-EXE/LogiCube/blob/Cours8/Bow/Voxel%20Weapons.zip)



Première étape : Creer Bow.cs

Ajouter ces variables d'instance

```
public KeyCode equipKey = KeyCode.B;
public GameObject bowPrefab;
public WorldManagerScript worldManager;
public CharacterController playerController;
public Vector3 bowLocalPosition = new Vector3(0.32f, -0.32f, 0.62f);
public float bowYRotation = -90f;
public Vector3 bowLocalScale = Vector3.one * 0.35f;
public Vector3 bowPullOffset = new Vector3(-0.10f, 0f, -0.07f);
public bool IsEquipped => isEquipped;
```

Étape 2 : Awake()

Ajouter ces variables d'instance

```
void Awake()  
{  
    PlayerAction playerAction = GetComponent<PlayerAction>();  
    if (playerAction != null)  
    {  
        if (worldManager == null) worldManager = playerAction.worldManager;  
        if (playerController == null) playerController = playerAction.playerController;  
    }  
}
```

- On récupère les refs depuis **PlayerAction** pour éviter de tout recâbler à la main.

Start()

```
void Start()
{
    bowVisual = Instantiate(bowPrefab, transform).transform;
    bowVisual.localPosition = bowLocalPosition;
    bowVisual.localEulerAngles = new Vector3(0f, bowYRotation, 0f);
    bowVisual.localScale = bowLocalScale;
    bowVisual.gameObject.SetActive(false);
    bowBaseLocalPosition = bowLocalPosition;
}
```

- On crée le modèle 3D de l'arc comme enfant de la caméra.
- Position/rotation/échelle = vue FPS correcte.
- `SetActive(false)` : l'arc est caché au début.

Update() + ToggleBow()

- B inverse l'état équipé/rangé.
- Si l'arc n'est pas équipé, on arrête la logique de tir.
- Quand on équipe, l'arc apparaît. Quand on range, il disparaît.

```
void Update()
{
    if (Input.GetKeyDown(equipKey))
        ToggleBow();

    if (!isEquipped)
        return;

    UpdatePull();
    UpdateVisual();
}

void ToggleBow()
{
    isEquipped = !isEquipped;
    pull = 0f;

    if (bowVisual != null)
        bowVisual.gameObject.SetActive(isEquipped);
}
```

UpdatePull()

- pull monte progressivement de 0 à 1.
- Relâcher la souris = tir.
- On remet pull à 0 après chaque tir.

```
void UpdatePull()
{
    if (Input.GetMouseButton(0))
        pull = Mathf.Clamp01(pull + Time.deltaTime * 4f);

    if (Input.GetMouseButtonUp(0))
    {
        Shoot();
        pull = 0f;
    }
}
```

UpdateVisual() (mini animation)

```
void UpdateVisual()
{
    if (bowVisual == null)
        return;

    bowVisual.localPosition = bowBaseLocalPosition + bowPullOffset * pull;
}
```

- Plus pull est haut, plus l'arc recule.
- C'est une animation simple mais efficace pour voir la charge.

Shoot() (spawn + vitesse)

```
void Shoot()  
{  
    GameObject arrow = GameObject.CreatePrimitive(PrimitiveType.Cylinder);  
    arrow.name = "Arrow";  
  
    arrow.transform.position = transform.position + transform.forward * 0.6f + transform.right * 0.12f - transform.up * 0.08f;  
    arrow.transform.rotation = Quaternion.FromToRotation(Vector3.up, transform.forward);  
    arrow.transform.localScale = new Vector3(0.025f, 0.35f, 0.025f);  
  
    Renderer renderer = arrow.GetComponent<Renderer>();  
    if (renderer != null)  
        renderer.material.color = new Color(0.30f, 0.18f, 0.08f);  
  
    Collider arrowCollider = arrow.GetComponent<Collider>();  
    if (arrowCollider != null && playerController != null)  
    {  
        Collider[] playerColliders = playerController.GetComponentsInChildren<Collider>();  
        for (int i = 0; i < playerColliders.Length; i++)  
            Physics.IgnoreCollision(arrowCollider, playerColliders[i], true);  
    }  
  
    Rigidbody rb = arrow.AddComponent<Rigidbody>();  
    float speed = Mathf.Lerp(16f, 28f, pull);  
    rb.velocity = transform.forward * speed;  
  
    ArrowProjectile arrowScript = arrow.AddComponent<ArrowProjectile>();  
    arrowScript.worldManager = worldManager;  
}
```

- On crée une flèche “à la volée”.
- On la place devant le joueur et orientée vers l’avant.
- La vitesse dépend de la charge (pull).

Créer ArrowProjectile.cs

- Nouveau Script dédié au comportement de la flèche.
- Sécurité : la flèche s'autodétruit au bout de 7 secondes.

```
public class ArrowProjectile : MonoBehaviour
{
    public WorldManagerScript worldManager;

    void Start()
    {
        Destroy(gameObject, 7f);
    }

    void OnCollisionEnter(Collision collision)
    {
        SlimeController slime = collision.transform.GetComponentInParent<SlimeController>();
        if (slime != null)
            slime.TakeHit();

        if (worldManager != null && collision.contactCount > 0)
        {
            ContactPoint contact = collision.GetContact(0);
            Vector3Int blockPos = Vector3Int.FloorToInt(contact.point - contact.normal * 0.01f);

            if (worldManager.Blocks.TryGetValue(blockPos, out BlockType blockType) && blockType == BlockType.Tnt)
                worldManager.Explode(blockPos);
        }

        Destroy(gameObject);
    }
}
```

Créer ArrowProjectile.cs

```
SlimeController slime = collision.transform.GetComponentInParent<SlimeController>();  
if (slime != null)  
|   slime.TakeHit();
```

- On vérifie si l'objet touché est (ou appartient à) un slime.
- Si oui, on applique un hit.

ArrowProjectile.cs

```
if (worldManager != null && collision.contactCount > 0)
{
    ContactPoint contact = collision.GetContact(0);
    Vector3Int blockPos = Vector3Int.FloorToInt(contact.point - contact.normal * 0.01f);

    if (worldManager.Blocks.TryGetValue(blockPos, out BlockType blockType) && blockType == BlockType.Tnt)
        worldManager.Explode(blockPos);
}
```

- On calcule la case voxel touchée.
- Si cette case est une TNT, on déclenche l'explosion.

ArrowProjectile.cs

```
if (worldManager != null && collision.contactCount > 0)
{
    ContactPoint contact = collision.GetContact(0);
    Vector3Int blockPos = Vector3Int.FloorToInt(contact.point - contact.normal * 0.01f);

    if (worldManager.Blocks.TryGetValue(blockPos, out BlockType blockType) && blockType == BlockType.Tnt)
        worldManager.Explode(blockPos);
}
```

- On calcule la case voxel touchée.
- Si cette case est une TNT, on déclenche l'explosion.

Intégration dans PlayerAction

- Quand l'arc est équipé, on bloque pose/casse de blocs.
- Ça évite conflit "clik gauche tire" vs "clik gauche casse"

```
[Header("Weapon")]
public Bow bow;

void Update()
{
    bool bowEquipped = bow != null && bow.IsEquipped;

    if (!bowEquipped)
        HandleBlockActions();

    HandleTntIgnite();
}
```

Configuration finale dans l'Inspector

Mettre exactement ces valeurs sur Main Camera

